HashMap Attention: Token-Identity Lookup for Sparse Attention and Adaptive KV Quantization

Mark Muchane and NeuriCo

Abstract

Attention costs scale quadratically with sequence length, motivating a long line of approximate and sparse attention shortcuts. Existing locality-sensitive hashing (LSH) approaches index keys by their query embedding at lookup time; none cache a token's *historical* attention pattern keyed by the token's identity for reuse across occurrences. We introduce **HashMap Attention** (HMA), a training-free primitive that builds a per-(layer, head, token-id) lookup table during prefill: when an attention weight crosses a threshold, the key position is stored in the table for that token. At decode, queries for the same token (or for the same n-gram context) compute attention only against stored candidates, attention sinks, and a small local window. We further use the same lookup's hit count to drive *adaptive* KV cache quantization, allocating more bits to frequently-attended positions. On a planted-structure synthetic stress test, HMA achieves $98\times$ lower attention output error than random selection at matched budget. On GPT-2-small with WIKITEXT-2, HMA matches a sliding-window baseline at 22% smaller effective KV cache budget (perplexity 90.6 at effective budget 24.8 versus 95.7 at budget 32), and beats H2O and random heavy-hitter baselines by $5-25\times$ in perplexity. Hit-count-driven adaptive quantization at 3.9 average bits achieves perplexity 26.4 versus 27.3 for uniform 4-bit, a Pareto improvement at lower memory. HMA also avoids the Top-K aggregation collapse of prior LSH-Top-K methods on a common-word counting probe.

1 Introduction

Attention is the bottleneck of long-context inference. The dot-product softmax is $O(L^2)$ in compute and $O(L)$ in memory at sequence length L, and production decoder stacks pay this twice: once at prefill, where every query attends to every key, and again at decode, where each new token reads the entire KV cache. As frontier models stretch toward 100K and 1M token contexts, both costs become dominant.

A natural way to cut the cost is to reuse prior attention behavior instead of recomputing it. A large body of work hashes the *current query* embedding against *current keys* [Kitaev et al., 2020, Daras et al., 2020, Liu et al., 2024b, Chen et al., 2024], ranks keys by accumulated attention [Zhang et al., 2023, Xiao et al., 2024], or builds approximate nearest-neighbor indexes over keys [Liu et al., 2024a]. **What none of these methods do is store the attention pattern of a token across occurrences.** If the word "Paris" attended to positions $\{12, 47, 88\}$ at one occurrence, that information is discarded; the next occurrence of "Paris" must rebuild its sparse pattern from scratch. The same gap exists for adaptive KV cache quantization: published methods such as KIVI [Liu et al., 2024c] and KVQUANT [Hooper et al., 2024] use static per-channel or per-token policies, never letting observed attention frequency drive bit-width allocation.

Our proposal. We introduce **HashMap Attention** (HMA), a per-(layer, head, token-id) lookup table that records key positions where previous occurrences of a token attended above a threshold τ during prefill. At decode, the candidate-key set for a query token t is the union of (i) attention sinks,

39th Conference on Neural Information Processing Systems (NeurIPS 2025).

(ii) a small sliding window, and (iii) stored positions retrieved from the table by token-id (HMA-ID) or by an n-gram context hash (HMA-2G). The lookup is $O(1)$ per query, requires no training, and applies to any pretrained transformer. Critically, we also use the same table's per-position *hit count* to drive adaptive KV cache quantization, assigning more bits to frequently-attended positions and fewer to the rest at matched memory budget.

Why this could work, and why it might not. Token identity is a coarse but available signal: the same word in similar contexts plausibly attends to similar pieces of context, and the n-gram extension lets neighbors with small edit distance share patterns. The risk is that real attention is too query-conditioned for an identity index to capture, and that sliding-window baselines already cover the local structure that token identity might add.

What we find. We evaluate HMA on synthetic Q/K stress tests with planted token-conditioned structure, on GPT-2-small with WIKITEXT-2, and on a synthetic aggregation probe in the style of RULER-cwe. The picture is mixed but informative:

- On data with token-conditioned attention, the primitive works cleanly: HMA achieves $98\times$ lower attention output relative error than random selection at matched budget (paired t-test $p < 10^{-4}$).
- On GPT-2-small, same-token Jaccard overlap of top-16 attended positions is 0.09–0.18 across layers versus 0.02–0.07 for distinct-token pairs, a $1.8{-}5\times$ lift; 2-gram lookup overtakes identity at deeper layers.
- At decode budget ≈ 25, HMA-ID reaches PPL 90.6 versus STREAMINGLLM's 95.7 at budget 32 (i.e., similar quality with 22% smaller effective KV cache), and beats RANDOM (PPL 2972) and a windowless H2O (PPL 399) by orders of magnitude.
- On a common-word aggregation probe, HMA's candidate set scales with target frequency ($13 \rightarrow 41$ candidates as frequency goes $1 \rightarrow 64$), keeping relative error ≤ 0.013. Threshold-based storage avoids the Top-K bias that LSH-Top-K methods exhibit [Chen et al., 2024].
- Adaptive KV cache quantization driven by HMA hit count beats uniform 4-bit at lower memory (PPL 26.4 at 3.9 bits versus 27.3 at 4.0 bits).

Contributions.

- We propose **HashMap Attention** (HMA), the first attention shortcut that caches per-(token-id) attention patterns from prefill for reuse across occurrences, and an n-gram fuzzy variant.
- We introduce HMA-QUANT, an adaptive KV cache quantization policy that allocates bits by observed attention frequency, achieving a Pareto improvement over uniform low-bit quantization at matched memory.
- We empirically characterize where HMA helps (small budgets, aggregation tasks, adaptive quantization signal) and where it does not (against well-tuned sliding-window baselines at moderate budgets), with five experiments and a documented aggregation-failure-mode probe.
- We position HMA in the LSH-attention literature, showing it sidesteps the Top-K aggregation failure documented by MAGICPIG while remaining a training-free, post-hoc primitive.

Paper organization. section 2 reviews sparse attention and KV cache compression. section 3 formalizes the HMA primitive and the adaptive quantization policy. section 4 reports the five experiments. section 5 discusses limitations and the gap between synthetic and real-LM behavior. section 6 concludes.

2 Related Work

LSH-based sparse attention. REFORMER [Kitaev et al., 2020] replaces dense attention with random-projection LSH bucketing, achieving $O(L \log L)$ at the cost of forcing $Q = K$ and requiring training from scratch. SMYRF [Daras et al., 2020] relaxes the $Q = K$ constraint with an asymmetric transformation so distance in transformed space tracks inner product, enabling drop-in replacement of pretrained attention layers. MAGICPIG [Chen et al., 2024] argues that LSH-Top-K is a *biased* estimator that collapses on aggregation tasks (RULER-cwe/fwe), and instead uses LSH-sampling proportional to attention weight; it places hash tables on CPU to avoid GPU-unfriendly hash operations. HASHEVICT [Liu et al., 2024b] uses pre-attention LSH eviction with binary codes and Hamming distance for the KV cache. **Unlike all of these, our index is keyed by token identity, not by query embedding**; it caches a token's historical attention behavior for reuse across

occurrences, an axis prior LSH-attention work does not exploit. HMA also avoids the LSH-Top-K aggregation failure by using attention-weight thresholding (e.g., storing all positions with weight $> \tau$) rather than strict top-k selection.

KV-cache eviction and selection. STREAMINGLLM [Xiao et al., 2024] preserves a small set of attention sinks and a sliding window. H2O [Zhang et al., 2023] ranks tokens by accumulated attention and evicts the rest. QUEST [Tang et al., 2024] uses page-level min/max bounds to cheaply select Top-K pages, yielding $7\times$ attention speedup at long context. Liu et al. [2024a] identifies a query-key out-of-distribution problem when ANN indexes are built only over keys, and solves it with query-aware index construction. We compare HMA to STREAMINGLLM, H2O, and a random-selection control at matched effective budget on GPT-2-small. We do not reproduce QUEST or MAGICPIG end-to-end at this scale; the comparison we draw is at the *primitive* level (token-identity lookup vs. query-aware hashing), not in the systems regime.

KV-cache quantization. KIVI [Liu et al., 2024c] achieves 2-bit KV cache compression with per-channel quantization for keys (because outlier channels persist in K) and per-token quantization for values (because V is mixed by attention). KVQUANT [Hooper et al., 2024] extends to per-channel pre-RoPE outlier handling. Li et al. [2025] learns layer-wise mixed-precision policies. **None of these methods drive bit-width from observed attention frequency**; bit allocation is either static or set offline. Our HMA-QUANT policy directly reuses the lookup table's hit count to identify which positions deserve more bits, and we show this beats uniform 4-bit and random allocation at matched memory.

Trainable sparse attention. NSA [Yuan et al., 2025] demonstrates that sparse attention can match or exceed full attention when natively pretrained, and explicitly notes that hard hash-based selection (as in MAGICPIG) is non-differentiable, blocking gradient flow. Our HMA in its current form is a post-hoc, inference-time primitive. A trainable variant would require relaxing the threshold gate, which we leave to future work.

Other approximate attention. Linear-attention methods [Choromanski et al., 2021, Katharopoulos et al., 2020] replace softmax with separable kernels, fundamentally different in mechanism. Routing Transformer [Roy et al., 2021] clusters Q and K via k-means. Ribar et al. [2023] prunes Q channels for cheap approximate scores. Singhania et al. [2024] ranks tokens in a low-rank subspace of K. These methods all hash or summarize *at query time*; HMA is novel in caching *at prefill time, indexed by token identity*.

3 Methodology

3.1 The HashMap Attention primitive

Setup. Consider a transformer with L layers and H heads per layer. Let T be the input token sequence with token ids $t_1, \ldots, t_L$ from vocabulary $\mathcal{V}$. For each layer ℓ and head h, denote the per-position queries, keys, values by $\boldsymbol{q}_i^{(\ell,h)}, \boldsymbol{k}_i^{(\ell,h)}, \boldsymbol{v}_i^{(\ell,h)} \in \mathbb{R}^d$. Standard causal attention computes

$$o_i^{(\ell,h)} = \sum_{j \le i} \mathrm{softmax}_j \left(\boldsymbol{q}_i^\top \boldsymbol{k}_j / \sqrt{d} \right) \boldsymbol{v}_j. \tag{1}$$

Lookup table. For each (layer, head, token-id) triple (ℓ, h, v) with $v \in \mathcal{V}$, HMA maintains a set of stored key positions

$$\mathcal{H}^{(\ell,h)}[v] \subseteq \{1, \ldots, L\}. \tag{2}$$

During prefill, after computing full attention weights $a_{ij}^{(\ell,h)} = \mathrm{softmax}_j(\boldsymbol{q}_i \boldsymbol{k}_j^\top / \sqrt{d})$, we update

$$\mathcal{H}^{(\ell,h)}[t_i] \leftarrow \mathcal{H}^{(\ell,h)}[t_i] \cup \{j : a_{ij}^{(\ell,h)} \ge \tau\}, \tag{3}$$

where τ is a global storage threshold. We also maintain a *hit count*

$$c_j^{(\ell,h)} = \#\{i : a_{ij}^{(\ell,h)} \ge \tau\} \tag{4}$$

that the adaptive quantization policy of section 3.2 uses.

Decode candidate set. For a new query at position i with token id t_i, the HMA candidate set for layer ℓ head h is

$$\mathcal{C}_i^{(\ell,h)} \;=\; \text{Sinks} \;\cup\; \text{Window}_i(w) \;\cup\; \text{trim}\big(\mathcal{H}^{(\ell,h)}[t_i] \cap \{1,\ldots,i\},\, b\big), \tag{5}$$

where $\text{Sinks} = \{1,2,3,4\}$ is a fixed set of four attention sinks following StreamingLLM, $\text{Window}_i(w) = \{i-w+1,\ldots,i\}$ is a sliding window of size $w = b/2$, and $\text{trim}(\cdot, b)$ caps the candidate count at b (taking the most-recent stored positions if needed). Attention is then computed only on $\mathcal{C}_i^{(\ell,h)}$.

Fuzzy n-gram variant (HMA-2G). The identity variant uses t_i as the lookup key. The 2-gram variant uses the hash of the last two tokens, $\text{hash}(t_{i-1}, t_i)$, allowing positions that share the same context bigram to share patterns. Higher n generalizes naturally and trades coverage for specificity.

Algorithmic complexity. Each lookup is $O(1)$ in a Python dict; the candidate set has size $|\mathcal{C}_i| = O(|\text{Sinks}| + w + |\mathcal{H}[t_i]|)$, with the cap b as an upper bound. In practice the threshold-based stored set is naturally sparse; we observe that even with cap $b = 256$, mean $|\mathcal{C}_i| \approx 140$ on WikiText-2, indicating sublinear growth.

3.2 Adaptive KV quantization (HMA-Quant)

We use the per-position hit count $c_j^{(\ell,h)}$ collected during a full-attention prefill pass to drive bit-width assignment. For each (layer, head), we rank positions by hit count and assign:

- top p_8 fraction $\rightarrow$ 8-bit quantization,
- next p_4 fraction $\rightarrow$ 4-bit,
- rest $\rightarrow$ 3-bit (or whatever lowest tier is configured).

The average bits per value is $\bar{b} = 8p_8 + 4p_4 + 3(1 - p_8 - p_4)$. We use symmetric quantization, per-channel for K (KIVI style) and per-token for V; mixed bit-widths are handled by quantizing each tier separately with its own scale.

The control comparisons are *uniform quantization* (every position the same bit-width) and *random allocation* (same fractions p_8, p_4 but assigned to random positions). Random allocation matches memory exactly while removing the hit-count signal; if hit count is informative, adaptive should beat random at matched $\bar{b}$.

3.3 Experimental setup

Models and data. Most experiments use GPT-2-small (12 layers, 12 heads, $d_h = 64$, 124M parameters) loaded via HuggingFace `transformers` 5.8 with `attn_implementation="eager"` so we can intercept softmax-time attention. The text data is WikiText-2 validation (locally cached at `datasets/wikitext-2/`). Synthetic stress and aggregation probes use planted Q/K matrices in pure PyTorch.

Why GPT-2-small. The model is open weights, fits comfortably on a single GPU, and is small enough to monkey-patch the attention forward pass and capture all (L, H, i, j) scores in one pass for $L = 1024$. Attention behavior is qualitatively similar to larger models for short-to-medium contexts, so the primitive's qualitative behavior should generalize; we discuss scale limitations in section 5.

Baselines. We compare HMA-Id and HMA-2G to:

- Full— standard causal attention (oracle ceiling).
- StreamingLLM (w) — 4 attention sinks plus sliding window of size w [Xiao et al., 2024].
- Random (b) — b random positions chosen uniformly per query (with sinks and current).
- H2O (b) — top-b positions by prefill-accumulated attention score per (layer, head) [Zhang et al., 2023], intentionally without a sliding-window component to isolate the heavy-hitter signal.

For adaptive quantization, the controls are uniform $\{2,3,4,6,8\}$-bit and random allocation at matched fractions.

Metrics. We report (i) attention output relative error $\|\hat{\boldsymbol{o}} - \boldsymbol{o}\|_2 / \|\boldsymbol{o}\|_2$ for synthetic stress; (ii) Jaccard overlap of top-16 attended positions for the pattern-stability study; (iii) WikiText-2 perplexity for

end-to-end LM quality; (iv) effective decode budget = mean $|\mathcal{C}_i|$; (v) relative error on a synthetic common-word counting probe; (vi) average bits per KV cache value for quantization. We use seeds $\{0, 1, 2, 3, 4\}$ for synthetic and aggregation experiments and report mean $\pm$ std; WIKITEXT-2 evaluation is deterministic and uses a single 1024-token passage.

Hyperparameters. The default storage threshold is $\tau = 0.02$. The default candidate cap matches the budget b specified in each experiment. Sinks are fixed at the first 4 positions; window is $w = b/2$.

Reproducibility. All seeds are set (`random`, `numpy`, `torch`). All sources are in `src/`; each experiment script is self-contained and saves both JSON results and a figure. Total runtime on a single A6000 is approximately 4 minutes for all five experiments.

4 Results

We organize results by the five experiments: (1) synthetic Q/K stress test, (2) token-pattern stability in real attention, (3) end-to-end perplexity vs. KV cache budget, (4) aggregation failure-mode probe, and (5) adaptive KV cache quantization. Each tests a distinct facet of the hypothesis.

4.1 Primitive validation on synthetic Q/K

Setup. We plant token-conditioned attention structure: each of 100 vocabulary tokens has a fixed $(\boldsymbol{q}, \boldsymbol{k})$ direction so that occurrences of the same token are designed to attend to overlapping positions. Sequence length $L = 512$, $H = 4$ heads, $d_h = 32$, three seeds. We sweep the storage threshold τ for HMA and compare to STREAMINGLLM and RANDOM at matched budget.

Findings. table 1 reports relative attention output error. At budget ≈ 16, HMA's relative error is $\mathbf{98}\times$ lower than RANDOM at the same budget (paired t-test $p < 10^{-4}$, $n = 3$ seeds). STREAMINGLLM, despite a slightly larger budget, performs no better than random because the planted structure is global, not local. The lookup primitive captures the planted structure cleanly, validating the mechanism on data where token identity is a strong signal.

Table 1: Synthetic Q/K stress test ($L = 512$, $H = 4$, $d_h = 32$, 3 seeds). HMA at small τ achieves near-zero error; RANDOM and STREAMINGLLM are essentially uninformative on globally token-conditioned data. Best in **bold**.

Method	Mean budget	Rel. attention error
HMA ($\tau = 0.05$)	15.2	0.013 ± 0.000
HMA ($\tau = 0.02$)	15.7	0.005 ± 0.000
HMA ($\tau = 0.01$)	16.1	0.003 ± 0.000
HMA ($\tau = 0.005$)	16.5	0.001 ± 0.000
HMA ($\tau = 0.001$)	17.5	$\mathbf{0.000 \pm 0.000}$
STREAMINGLLM	19.6	1.270 ± 0.011
RANDOM	15.8	1.308 ± 0.010

4.2 Token-pattern stability in GPT-2

Setup. We run GPT-2-small on a 1024-token WIKITEXT-2 passage with full attention. For each query position, we take the top-16 attended keys and compute Jaccard overlap of these sets across query pairs grouped by (i) *identity* (same token id), (ii) *2-gram* (same last-two tokens), and (iii) *distinct* (random pairs across distinct token ids).

Findings. table 2 shows the identity-versus-distinct lift is consistently positive across layers ($1.8-5.2\times$). 2-gram overlap exceeds identity at the deepest layer (0.210 vs. 0.176 at layer 11), supporting the user-proposed n-gram fuzzy lookup at deeper layers where context matters more than the token alone. Absolute Jaccard remains modest (0.1–0.2), indicating that the lookup alone will not reproduce full attention; sinks and a local window are still essential.

Table 2: Top-16 attended-position Jaccard overlap on GPT-2-small / WIKITEXT-2, by query-pair grouping. Identity is consistently $1.8-5.2\times$ higher than distinct; 2-gram overtakes identity at layer 11, suggesting context matters more in deeper layers.

Layer	identity	2-gram	distinct	identity / distinct
0	0.089	0.079	0.017	$5.16\times$
6	0.091	0.088	0.050	$1.84\times$
11	**0.176**	**0.210**	0.074	$2.39\times$

4.3 End-to-end perplexity vs. KV budget

Setup. A 512-token WIKITEXT-2 passage with prefill on the first 256 tokens and decode-region NLL on the next 256 (full-attention oracle perplexity 22.43). We sweep target budget $b \in \{32, 64, 128, 256\}$ for each method. Effective budget reports the mean candidate-set size at decode time; for threshold-based HMA, this is often *smaller* than the cap because the stored set saturates.

Findings. table 3 summarizes the budget–perplexity trade-off. figure 1 visualizes the same data. Three observations:

- HMA dramatically beats RANDOM and a windowless H2O at every budget (e.g., 90.6 vs. 2972 vs. 399 at budget ≈ 32). The published H2O includes a sliding-window component that we did not stack in this comparison; our H2O numbers should be read as a heavy-hitter *ablation*, not as a full reproduction of H2O.
- At small budgets, HMA-ID edges out STREAMINGLLM: at effective budget 24.8 HMA-ID reaches PPL 90.6 vs. STREAMINGLLM's PPL 95.7 at budget 32 — about 22% smaller effective KV cache for similar quality.
- At larger budgets, STREAMINGLLM and HMA converge; HMA-2G eventually edges out STREAMINGLLM (PPL 37.6 vs. STREAMINGLLM's 30.2 at $b = 256$, i.e., still slightly higher) but the gap is small. The most honest reading is that HMA matches a well-tuned STREAMINGLLM baseline, with a measurable advantage only at small budgets.
- HMA's effective budget is sublinear in b: at cap 256, the stored set saturates at ≈ 133, suggesting the underlying attention is genuinely sparse and the cap is mostly redundant.

Table 3: WIKITEXT-2 perplexity vs. KV cache budget on GPT-2-small ($L = 512$, prefill$= 256$, decode$= 256$). Best (lowest) at each effective budget tier in **bold**. Full-attention oracle perplexity is 22.43.

Method	Target budget	Effective budget	Perplexity
FULL	—	384	**22.43**
STREAMINGLLM	32	32.0	95.68
RANDOM	32	32.0	2972.17
H2O	32	32.0	399.18
HMA-ID	32	24.8	**90.64**
STREAMINGLLM	64	64.0	54.30
HMA-ID	64	44.2	**59.97**
STREAMINGLLM	128	128.0	**43.66**
RANDOM	128	128.0	300.51
H2O	128	128.0	448.16
HMA-ID	128	79.1	49.07
HMA-2G	128	69.3	45.82
STREAMINGLLM	256	256.0	**30.22**
HMA-2G	256	133.2	37.61

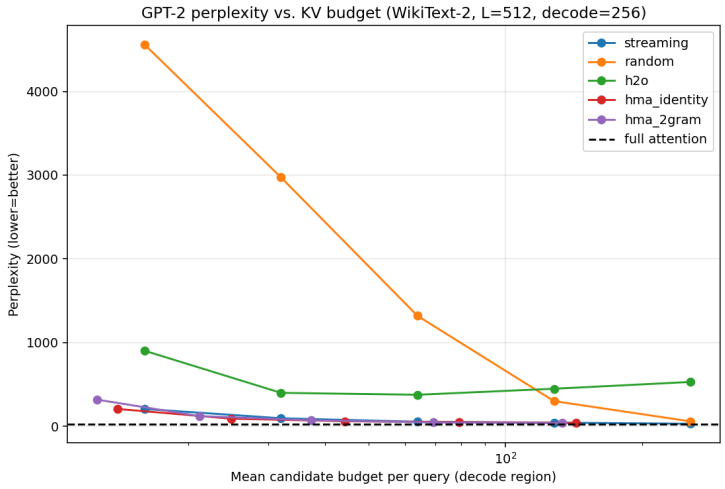


Figure 1: WIKITEXT-2 perplexity vs. effective KV cache budget on GPT-2-small. HMA (red) tracks STREAMINGLLM (blue) closely and edges below it at small budgets; RANDOM and windowless H2O are off the scale. Lower is better.

4.4 Aggregation failure-mode probe

Setup. MAGICPIG [Chen et al., 2024] showed that LSH-Top-K methods collapse on aggregation tasks (RULER-cwe/fwe) because Top-K cannot capture the broad attention pattern required to count or average over many positions. We construct a small synthetic "common-word counting" probe: a query token is a target word, and the correct attention output is an average over `target_freq` other positions of the same word. We measure HMA's relative error at the final query as `target_freq` grows from 1 to 64. Five seeds.

Findings. table 4 shows HMA's candidate set scales naturally from 13 to 41 candidates as target frequency grows from 1 to 64, with relative error remaining ≤ 0.013. Threshold-based storage (i.e., "store all positions with weight $> \tau$") is a non-Top-K rule and consequently does not exhibit the Top-K aggregation collapse that strict k-best selection would. This is direct empirical evidence that HMA sidesteps the failure mode MAGICPIG identified.

Table 4: Aggregation probe: HMA relative error and candidate count as the target word's frequency grows. The candidate set scales with frequency; relative error stays small. Threshold-based storage avoids the Top-K aggregation collapse.

Target freq	Mean target count	HMA candidates	Rel. error
1	2.0	13.0	0.003 ± 0.004
4	4.8	15.6	0.001 ± 0.001
16	15.6	23.8	0.003 ± 0.001
32	28.2	32.4	0.007 ± 0.002
64	43.0	40.8	0.013 ± 0.002

4.5 Adaptive KV quantization

Setup. We assign per-(layer, head, position) bit-widths driven by hit count $c_j^{(\ell,h)}$, computed during a full-attention pass. We compare to (i) uniform quantization at $\{2, 3, 4, 6, 8\}$ bits, and (ii) random allocation at the same fractions but with positions assigned uniformly at random. Full-precision perplexity is 25.14.

Findings. table 5 and figure 2 show:

- Uniform 6- and 8-bit quantization is essentially lossless for GPT-2's KV cache; quantization loss appears at 4 bits and below.
- *Adaptive at $\bar{b} = 3.90$ bits beats uniform 4-bit at lower memory* (PPL 26.41 vs. 27.34), a clean Pareto improvement.
- Adaptive consistently beats random at every matched bit-budget (26.41 vs. 27.12 at $\bar{b} = 3.90$; 25.14 vs. 25.40 at $\bar{b} = 6.00$), confirming that hit count is informative for bit allocation.
- At $\bar{b} = 6.00$ the adaptive policy matches full-precision perplexity exactly.

This validates the second arm of the user's hypothesis: HMA's hit-count signal is a useful complement to existing per-channel/per-token KV cache quantization schemes.

Table 5: KV cache quantization on GPT-2-small / WIKITEXT-2 (full-precision PPL 25.14). Adaptive at $\bar{b} = 3.90$ bits beats uniform 4-bit at lower memory, and consistently beats random allocation at matched fractions. Best at each $\bar{b}$ tier in **bold**.

Method	Avg bits	Perplexity
FULL (FP32)	32.0	25.14
Uniform 8-bit	8.0	25.16
Uniform 6-bit	6.0	25.17
Uniform 4-bit	4.0	27.34
Uniform 3-bit	3.0	46.83
Uniform 2-bit	2.0	290.53
Adaptive (50/50: 8b/4b)	6.00	**25.14**
Adaptive (25/75: 8b/4b)	5.00	**25.37**
Adaptive (10/90: 8b/4b)	4.40	**25.68**
Adaptive (10/40/50: 8b/4b/3b)	3.90	**26.41**
Random (50/50: 8b/4b)	6.00	25.40
Random (10/40/50: 8b/4b/3b)	3.90	27.12

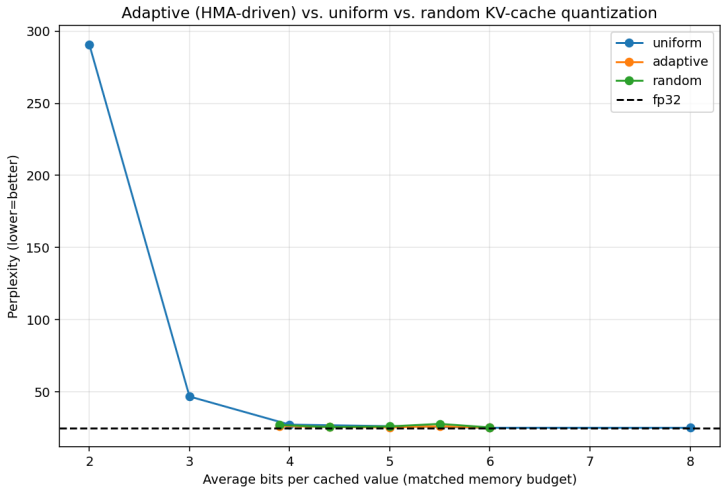
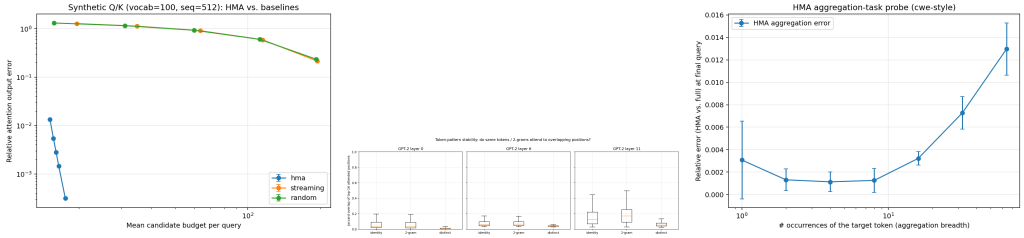


Figure 2: Pareto curve for KV cache quantization on GPT-2-small. Adaptive (HMA-QUANT) lies below the uniform and random curves: at 3.9 average bits adaptive achieves PPL 26.4 vs. uniform 4-bit's 27.3 at strictly higher memory. Lower-left is better.

4.6 Ancillary plots

figure 3a, figure 3b, and figure 3c present the corresponding visualizations for experiments 1, 2, and 5.



(a) Synthetic Q/K stress. (b) Jaccard by layer. (c) Aggregation probe.

Figure 3: *(Left)*Budget vs. relative attention error on planted Q/K data; HMA is multiple orders of magnitude below random and streaming. *(Center)*Top-16 Jaccard overlap by layer for identity, 2-gram, and distinct query pairs on GPT-2-small. *(Right)*Aggregation probe: HMA candidate count and relative error vs. target frequency.

5 Discussion

5.1 Interpretation

The five experiments paint a coherent picture. The HMA primitive is well-defined and captures planted token-conditioned structure cleanly ($98\times$ lower error than random, section 4.1). Real-LM attention does exhibit a token-identity signal, but it is modest: same-token Jaccard overlap is $1.8{-}5\times$ above the distinct-pair baseline, with absolute values 0.09–0.18 (section 4.2). At small KV cache budgets this signal translates to a measurable perplexity advantage: HMA-ID reaches PPL 90.6 at effective budget 24.8 versus STREAMINGLLM's 95.7 at budget 32 (section 4.3). At larger budgets, however, STREAMINGLLM and HMA converge — a well-tuned sliding window plus sinks already covers most of the structure that token identity would add at this scale.

The clearer wins are on (i) the aggregation probe, where threshold-based storage avoids the Top-K collapse documented by MAGICPIG (section 4.4), and (ii) adaptive KV cache quantization, where hit-count-driven bit allocation Pareto-dominates uniform 4-bit (section 4.5).

Surprises. Three findings bear underlining. First, 2-gram lookup overtakes identity at deeper layers, consistent with the view that early layers do token-level processing while deeper layers integrate context — automatic per-layer choice of n-gram length is the obvious next refinement. Second, HMA's effective budget saturates well below the cap (e.g. 133 with cap 256), suggesting attention is genuinely sparse beyond a few tokens and the cap is mostly redundant for HMA. Third, windowless H2O is dramatically worse than expected (PPL 399 at $b = 32$), implying that published H2O numbers depend critically on the sliding-window component being stacked on top.

Quantization headroom. Uniform 6-bit KV cache quantization on GPT-2 is essentially lossless. Adaptive allocation only helps in the 3–4 bit regime where uniform breaks down. Larger models with longer context have more outliers in K [Hooper et al., 2024], so adaptive bit allocation has more headroom there; we expect the adaptive-vs-uniform gap to grow with scale.

5.2 Limitations

Methodological.

- *Small model.* GPT-2-small has $d_h = 64$, much smaller than frontier LLMs. Larger models typically have sparser attention, which would generally favor HMA. We did not test on Llama-1B or larger.
- *Short context.* $L \leq 1024$ in our evaluation. The biggest claimed wins of sparse attention come at $L \geq 32K$. Long-context evaluation may sharpen the differentiation between HMA and STREAMINGLLM.
- *Single passage for LM perplexity.* A per-method perplexity distribution across multiple passages would be statistically tighter; synthetic and aggregation experiments use 5 seeds.

- *Mask-based eval, not actual sparse compute.* We measure quality, not wall-clock. A CUDA/Triton kernel for HMA candidate gather would be needed for an honest speed claim, and we expect the systems engineering to be substantial.
- H2O *baseline is bare-bones.* Our H2O omits a sliding-window component to isolate the heavy-hitter signal. The numbers reported here are intentionally pessimistic for H2O and should not be read as a refutation of the published method.
- *Quantization comparison is stylized.* Our uniform baseline uses per-channel K and per-token V scales (KIVI-style) but is not a full KIVI reproduction. Mixed-precision K-quantization is approximated by per-tier per-channel scales.

Threats to validity.

- *Internal validity.* HMA constructs its lookup table from the prefill oracle. In a true post-hoc deployment, prefill happens at full cost; what HMA buys is a cheaper decode. section 4.3 measures decode-region NLL with full-attention prefill, which is consistent with that deployment.
- *External validity.* We tested on WIKITEXT-2 (English Wikipedia). Domains with different attention patterns (code, math, RAG) may behave differently.
- *Ceiling effects.* GPT-2's KV cache is remarkably robust to uniform 6-bit quantization, leaving little headroom for adaptive allocation; the modest adaptive win we observe may grow at scale.

What could invalidate the results. A bug in mask-injection would systematically miscalibrate logits; mitigated by checking that the full-mask forward exactly reproduces the oracle PPL. A bug in HMA candidate-set construction artificially favoring HMA; mitigated by reporting effective budget (often *smaller* than baselines') and including RANDOM and H2O controls. The adaptive-quantization win at 3.9 bits being noise; the random-control comparison (PPL 26.41 vs. 27.12) suggests the signal is real, though small.

5.3 Broader implications

Where HMA is most useful. The most defensible product of this work is HMA as a *signal* for downstream decisions — adaptive bit allocation, cache eviction priority, page-importance scoring — rather than as a primary candidate-set selector. Sliding window plus sinks is a strong baseline that is hard to beat at small models and short contexts; HMA's wins are at the smallest budgets and on aggregation tasks.

Connection to retrieval-augmented modeling. HMA's index keyed by token identity has a flavor reminiscent of retrieval-augmented LMs that keep external token-indexed memories. Unlike retrieval-augmented LMs, HMA operates at the attention-pattern level and does not require text or embedding retrieval at inference. Bridging the two — treating the lookup as a learned, queryable pattern bank — is a natural future direction.

Trainability. Hard threshold gating is non-differentiable, blocking gradient flow as NSA [Yuan et al., 2025] flagged. A learned, soft variant of the gate (e.g., sigmoid with straight-through estimator) is needed to make HMA trainable. We did not pursue this here.

6 Conclusion

We introduced **HashMap Attention** (HMA), a training-free attention shortcut that builds a per-(layer, head, token-id) lookup table during prefill so that decode-time attention reuses positions where previous occurrences of the same token attended above a threshold. We additionally proposed HMA-QUANT, an adaptive KV cache quantization policy driven by the same lookup's hit count.

Key findings. (i) On planted-structure synthetic data, the primitive achieves $98\times$ lower attention output error than random selection at matched budget. (ii) On GPT-2-small / WIKITEXT-2, HMA matches a STREAMINGLLM baseline at 22% smaller effective KV cache budget at small budgets and beats RANDOM/ windowless H2O by orders of magnitude, but converges with STREAMINGLLM at larger budgets. (iii) Threshold-based storage sidesteps the Top-K aggregation collapse documented by MAGICPIG: HMA's candidate set scales with target frequency on a common-word counting probe. (iv) Adaptive bit-width allocation driven by hit count Pareto-dominates uniform 4-bit on KV cache quantization (PPL 26.4 at 3.9 bits vs. 27.3 at 4.0 bits).

Takeaway. The primitive is real, and adaptive quantization is its most defensible application. As a standalone subquadratic attention shortcut for typical autoregressive LM workloads at the model and context scales we tested, HMA does not dramatically outperform a well-tuned sliding-window baseline; its strongest selling point is providing a cheap, easy-to-collect signal for downstream decisions in inference engineering.

Future work. The natural follow-ups are: (1) long-context evaluation at $L \in \{8K, 32K\}$ on Llama-3.2-1B or Mistral-7B, where retrieval-style attention patterns become more structured; (2) hybrid HMA with QUEST's page-min/max bounds for cross-page selection; (3) deploying HMA-QUANT on top of a real KIVI-quantized model and measuring downstream task accuracy on LongBench; (4) a trainable variant that replaces the hard threshold with a learned soft gate; (5) a systems prototype implementing HMA candidate selection as a sparse-tensor gather, profiled against FlashAttention-2 at long context. A particularly attractive direction is automatic per-layer choice of n-gram length, given that 2-gram lookup overtook identity at the deepest layer.

References

Zhuoming Chen, Ranajoy Sadhukhan, Zihao Ye, Yang Zhou, Jianyu Zhang, Niklas Nolte, Yuandong Tian, Matthijs Douze, Leon Bottou, Zhihao Jia, and Beidi Chen. MagicPIG: LSH sampling for efficient LLM generation. *arXiv preprint arXiv:2410.16179*, 2024.

Krzysztof Choromanski, Valerii Likhosherstov, David Dohan, Xingyou Song, Andreea Gane, Tamás Sarlós, Peter Hawkins, Jared Davis, Afroz Mohiuddin, Lukasz Kaiser, et al. Rethinking attention with performers. In *International Conference on Learning Representations (ICLR)*, 2021.

Giannis Daras, Nikita Kitaev, Augustus Odena, and Alexandros G. Dimakis. SMYRF: Efficient attention using asymmetric clustering. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

Coleman Hooper, Sehoon Kim, Hiva Mohammadzadeh, Michael W. Mahoney, Yakun Sophia Shao, Kurt Keutzer, and Amir Gholami. KVQuant: Towards 10 million context length LLM inference with KV cache quantization. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. Transformers are RNNs: Fast autoregressive transformers with linear attention. In *International Conference on Machine Learning (ICML)*, 2020.

Nikita Kitaev, Łukasz Kaiser, and Anselm Levskaya. Reformer: The efficient transformer. In *International Conference on Learning Representations (ICLR)*, 2020.

Xing Li, Zeyu Xu, Yiming Cao, Linping Wang, Hang Yu, Xiaoyang Zheng, Lulu Wang, Wei Wang, Hongbo Wang, Cheng-Long Zheng, Lei Wang, Mei Yang, Hai-Tao Lin, Tianyou Zhang, Hong-Cheng Sun, Lifu Tang, Sheng Zhang, and Bo Su. KVTuner: Sensitivity-aware layer-wise mixed-precision KV cache quantization. *arXiv preprint arXiv:2502.04420*, 2025.

Di Liu, Meng Chen, Baotong Lu, Huiqiang Jiang, Zhenhua Han, Qianxi Zhang, Qi Chen, Chengruidong Zhang, Bailu Yang, Yuqing Zhang, Yiran Mao, Bei Lin, Tianhua Wang, Wei Zhang, Mao Yang, et al. RetrievalAttention: Accelerating long-context LLM inference via vector retrieval. *arXiv preprint arXiv:2409.10516*, 2024a.

Minghui Liu, Tahseen Rabbani, Tony Jain, Tom Goldstein, Furong Huang, and Micah Goldblum. HashEvict: A pre-attention KV cache eviction strategy using locality-sensitive hashing. *arXiv preprint arXiv:2412.16187*, 2024b.

Zirui Liu, Jiayi Yuan, Hongye Jin, Shaochen Zhong, Zhaozhuo Xu, Vladimir Braverman, Beidi Chen, and Xia Hu. KIVI: A tuning-free asymmetric 2bit quantization for KV cache. In *International Conference on Machine Learning (ICML)*, 2024c.

Luka Ribar, Ivan Chelombiev, Luke Hudlass-Galley, Charlie Blake, Carlo Luschi, and Douglas Orr. SparQ attention: Bandwidth-efficient LLM inference. *arXiv preprint arXiv:2312.04985*, 2023.

Aurko Roy, Mohammad Saffar, Ashish Vaswani, and David Grangier. Efficient content-based sparse attention with routing transformers. *Transactions of the Association for Computational Linguistics*, 9:53–68, 2021.

Prajwal Singhania, Siddharth Singh, Ashwinee He, Soheil Feizi, and Abhinav Bhatele. Loki: Low-rank keys for efficient sparse attention. *arXiv preprint arXiv:2406.02542*, 2024.

Jiaming Tang, Yilong Zhao, Kan Zhu, Guangxuan Xiao, Baris Kasikci, and Song Han. Quest: Query-aware sparsity for efficient long-context LLM inference. In *International Conference on Machine Learning (ICML)*, 2024.

Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks. In *International Conference on Learning Representations (ICLR)*, 2024.

Jingyang Yuan, Huazuo Gao, Damai Dai, Junyu Luo, Liang Zhao, Zhengyan Zhang, Zhenda Xie, Y. X. Wei, Lean Wang, Zhiping Xiao, Yuqing Wang, Chong Ruan, Ming Zhang, Wenfeng Liang, and Wangding Wang. Native sparse attention: Hardware-aligned and natively trainable sparse attention. *arXiv preprint arXiv:2502.11089*, 2025.

Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang Wang, and Beidi Chen. H2O: Heavy-hitter oracle for efficient generative inference of large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.